

# Port Wrangler — Complete Project Roadmap & Build Guide

A unified local developer infrastructure platform: databases, message queues, and AI model management in one tool — powered by Spring AI, RAG pipelines, and agentic tool calling.

---

## Table of Contents

1. [Project Vision](#)
  2. [Final Architecture](#)
  3. [Technology Stack](#)
  4. [Database Schema Evolution](#)
  5. [Phase 0 — Project Hardening](#)
  6. [Phase 1 — Kafka Module](#)
  7. [Phase 2 — LLM Runtime Manager](#)
  8. [Phase 3 — Spring AI Foundation](#)
  9. [Phase 4 — RAG Pipeline](#)
  10. [Phase 5 — Agentic Infrastructure Assistant](#)
  11. [Phase 6 — Polish & Community Launch](#)
  12. [API Design Reference](#)
  13. [Frontend Architecture](#)
  14. [Testing Strategy](#)
  15. [CI/CD Pipeline](#)
  16. [Resume Showcase Guide](#)
- 

## 1. Project Vision

Port Wrangler starts as a Docker-based database deployer and evolves into a **unified local infrastructure assistant** — the single tool a Java/backend developer opens before writing a single line of application code.

### The problem it solves

A developer starting a new microservices project currently needs to:

- Manually `(docker run)` Postgres, Redis, Kafka with the right flags they look up every time
- Open separate UI tools for each service (DBeaver, kafka-ui, Ollama)
- Copy connection strings from Stack Overflow
- Manually configure their AI model for local development
- Debug container failures by grepping raw logs

Port Wrangler replaces all of that with one tool — and adds an AI assistant that understands your entire local stack and can operate it for you.

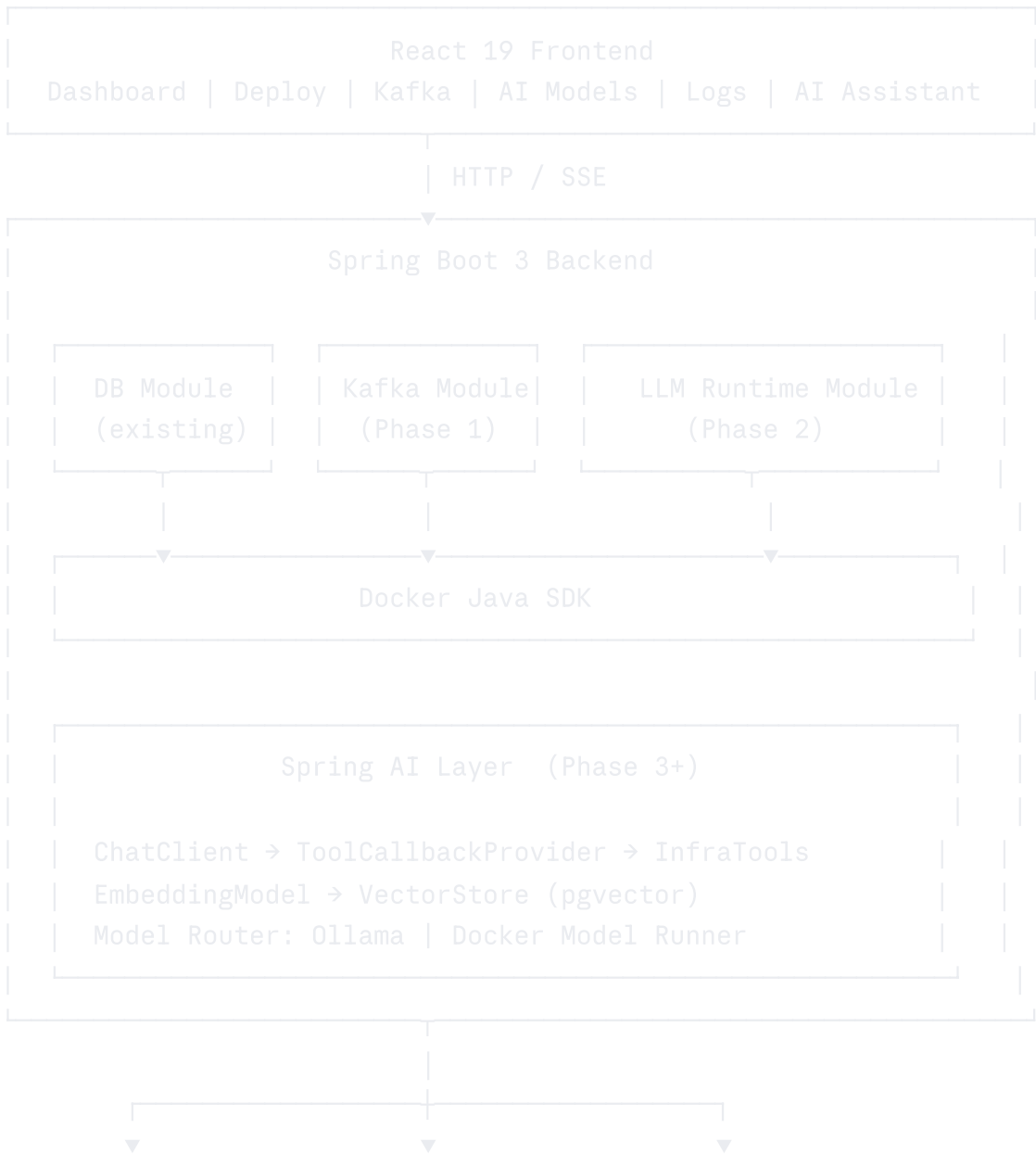
End-state vision (what Phase 5 looks like)

User: "Set up my standard microservices stack – Postgres 16 for orders, Redis for cache, Kafka for events, and load Llama 3.2 for local AI work"

Port Wrangler AI: [calls deployDatabase x2, deployKafka, deployModel]

"Done. Here are your Spring Boot configs for all four services. Your Kafka consumer lag is currently zero and all containers are healthy."

2. Final Architecture



|                |                 |               |
|----------------|-----------------|---------------|
| PostgreSQL 16  | pgvector ext    | Docker daemon |
| (system DB)    | (same instance) | (unix socket) |
| metadata store | vector store    | container ops |

## Key design principles

**Single PostgreSQL instance for everything.** The system DB you already have gets pgvector added as an extension. No new services needed for the vector store. One less thing for users to manage.

**Existing service layer as AI tools.** `DbInstanceService`, `KafkaService`, and `AiRuntimeService` are exposed as Spring AI `@Tool` methods without any new business logic. The AI goes through the same code paths as the UI. No dual maintenance.

**SSE streaming from day one.** The chat endpoint streams tokens. This is infrastructure you build in Phase 3 and reuse forever. Getting it right once is better than retrofitting it later.

## 3. Technology Stack

### Backend (final)

| Technology      | Version | Purpose           | Why                                                      |
|-----------------|---------|-------------------|----------------------------------------------------------|
| Java            | 21      | Language          | Records, virtual threads, pattern matching               |
| Spring Boot     | 3.4.x   | Framework         | Your existing choice, Spring AI requires 3.x             |
| Spring AI       | 1.0.x   | AI abstraction    | Tool calling, RAG, ChatClient, EmbeddingModel            |
| Spring Data JPA | managed | ORM               | Existing choice                                          |
| Spring WebFlux  | managed | SSE streaming     | Reactive streaming for chat responses                    |
| PostgreSQL      | 16      | System DB         | Existing choice                                          |
| pgvector        | 0.7.x   | Vector store      | Extension on existing Postgres — no new service          |
| Docker Java SDK | 3.4.x   | Docker ops        | Existing choice                                          |
| Flyway          | 9.x     | DB migrations     | Replace <code>ddl-auto: update</code> — production grade |
| Micrometer      | managed | Metrics           | Container and model metrics                              |
| Testcontainers  | 1.19.x  | Integration tests | Test Docker interactions realistically                   |

### Frontend (final)

| Technology        | Version | Purpose                         |
|-------------------|---------|---------------------------------|
| React             | 19      | UI framework                    |
| Vite              | 8       | Build tool                      |
| TailwindCSS       | 4       | Styling                         |
| React Query       | 5.x     | Server state, cache, polling    |
| Zustand           | 4.x     | Client UI state                 |
| React Router DOM  | 7       | Routing                         |
| Recharts          | 2.x     | Metrics charts                  |
| React Markdown    | 9.x     | Render AI responses as markdown |
| EventSource / SSE | native  | Stream chat tokens              |

New dependencies to add (build.gradle.kts)

```
kotlin

// Spring AI BOM – add to dependency management
implementation(platform("org.springframework.ai:spring-ai-bom:1.0.0"))

// Spring AI core
implementation("org.springframework.ai:spring-ai-spring-boot-autoconfigure")

// Ollama chat + embedding
implementation("org.springframework.ai:spring-ai-ollama-spring-boot-starter")

// pgvector store
implementation("org.springframework.ai:spring-ai-pgvector-store-spring-boot-starter")

// WebFlux for SSE
implementation("org.springframework.boot:spring-boot-starter-webflux")

// Flyway for migrations
implementation("org.flywaydb:flyway-core")
implementation("org.flywaydb:flyway-database-postgresql")
```

## 4. Database Schema Evolution

### Immediate change: Replace ddl-auto with Flyway

Your current `(ddl-auto: update)` is fine for development but risky long term. Switch to Flyway before adding any new tables. Existing tables become your baseline migration.

**Step 1:** Set `(spring.jpa.hibernate.ddl-auto: validate)`

**Step 2:** Create `(V1__baseline.sql)` by exporting your current schema.

**Step 3:** All future schema changes are versioned migration files.

### New tables by phase

#### Phase 1 — Kafka

```
sql

-- V2__kafka_instances.sql

CREATE TABLE kafka_instances (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(255) NOT NULL UNIQUE,
  version      VARCHAR(50) NOT NULL,
  broker_port  INT          NOT NULL,
  zookeeper_port INT,
  kraft_mode   BOOLEAN      NOT NULL DEFAULT true,
  status       VARCHAR(50) NOT NULL DEFAULT 'STOPPED',
  container_id VARCHAR(255),
  created_at   TIMESTAMP    NOT NULL DEFAULT now(),
  updated_at   TIMESTAMP    NOT NULL DEFAULT now()
);

CREATE TABLE kafka_topics (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  kafka_instance_id UUID NOT NULL REFERENCES kafka_instances(id) ON DELETE CASCADE,
  topic_name   VARCHAR(512) NOT NULL,
  partitions   INT NOT NULL DEFAULT 1,
  replication_factor INT NOT NULL DEFAULT 1,
  retained_ms  BIGINT,
  created_at   TIMESTAMP NOT NULL DEFAULT now()
);
```

#### Phase 2 — LLM Runtime

```
sql
```

```
-- V3__ai_runtimes.sql

CREATE TABLE ai_runtimes (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(255) NOT NULL UNIQUE,
  runtime_type VARCHAR(50) NOT NULL, -- OLLAMA, DOCKER_MODEL_RUNNER, LOCAL_AI
  host_port   INT          NOT NULL,
  status      VARCHAR(50) NOT NULL DEFAULT 'STOPPED',
  container_id VARCHAR(255),
  gpu_enabled BOOLEAN      NOT NULL DEFAULT false,
  gpu_type    VARCHAR(50), -- NVIDIA_CUDA, AMD_ROCM, APPLE_METAL
  created_at  TIMESTAMP    NOT NULL DEFAULT now()
);

CREATE TABLE ai_models (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  runtime_id  UUID          NOT NULL REFERENCES ai_runtimes(id) ON DELETE CASCADE,
  model_name  VARCHAR(255) NOT NULL, -- llama3.2:3b, gemma3:4b etc
  display_name VARCHAR(255),
  size_bytes  BIGINT,
  pull_status VARCHAR(50) NOT NULL DEFAULT 'PENDING',
  is_active   BOOLEAN      NOT NULL DEFAULT false,
  context_window INT,
  created_at  TIMESTAMP    NOT NULL DEFAULT now(),
  UNIQUE (runtime_id, model_name)
);

CREATE TABLE model_pull_log (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  model_id    UUID          NOT NULL REFERENCES ai_models(id) ON DELETE CASCADE,
  status      VARCHAR(50) NOT NULL,
  progress    INT,
  message     TEXT,
  logged_at   TIMESTAMP    NOT NULL DEFAULT now()
);
```

## Phase 3 & 4 — Spring AI + pgvector

sql

```
-- V4__pgvector_and_chat.sql

-- Enable pgvector extension
CREATE EXTENSION IF NOT EXISTS vector;

-- Spring AI managed table (autoconfigured but define explicitly for control)
CREATE TABLE vector_store (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    content     TEXT,
    metadata    JSONB,
    embedding   vector(768) -- dimension matches your embedding model
);

CREATE INDEX ON vector_store USING ivfflat (embedding vector_cosine_ops)
    WITH (lists = 100);

-- Chat session history
CREATE TABLE chat_sessions (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title       VARCHAR(512),
    created_at  TIMESTAMP NOT NULL DEFAULT now(),
    updated_at  TIMESTAMP NOT NULL DEFAULT now()
);

CREATE TABLE chat_messages (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id  UUID NOT NULL REFERENCES chat_sessions(id) ON DELETE CASCADE,
    role        VARCHAR(20) NOT NULL, -- USER, ASSISTANT, SYSTEM, TOOL
    content     TEXT,
    tool_calls  JSONB,
    msg_order   INT NOT NULL,
    created_at  TIMESTAMP NOT NULL DEFAULT now()
);

-- RAG document sources
CREATE TABLE rag_documents (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    source_type  VARCHAR(50) NOT NULL, -- DEPLOYMENT_LOG, CONTAINER_LOG, MANUAL
    source_id    VARCHAR(255),
    chunk_index  INT NOT NULL DEFAULT 0,
    content      TEXT NOT NULL,
    indexed_at   TIMESTAMP NOT NULL DEFAULT now()
);
```

## Phase 0 — Project Hardening (Weeks 1-2)

This phase produces no new features. It makes the existing project production-grade and contribution-ready. Do not skip it — technical debt now compounds against every future phase.

### 0.1 Rename the repository

`jubilant-doodle` is a GitHub auto-generated name. Rename to `port-wrangler`. Go to Settings → Repository name. Update all internal references.

### 0.2 Add a proper README header

Your current README is thorough but text-heavy. Add at the top:

- A short one-line description
- A GIF or screenshot of the deploy pipeline in action
- A "Quick Start" section that is literally three commands
- Badges: build status, license, Java version, latest release

The first 10 seconds of a visitor's experience determines if they star or leave.

### 0.3 Replace ddl-auto with Flyway

As described in the schema section. This is non-negotiable before adding new tables. A migration file you can point to in an interview is evidence of engineering maturity.

```
yaml
# application.yml changes
spring:
  jpa:
    hibernate:
      ddl-auto: validate      # was: update
  flyway:
    enabled: true
    locations: classpath:db/migration
    baseline-on-migrate: true # handles existing schema
```

### 0.4 Write integration tests for the existing deploy pipeline

You have no tests visible in the repo. Before extending the codebase, cover the core pipeline with Testcontainers-based integration tests.

```
java
```



```

@SpringBootTest
@Testcontainers
class DeploymentPipelineIntegrationTest {

    @Container
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine");

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired
    private DbInstanceService service;

    @Test
    void deploymentPipelineCompletesAllFourSteps() {
        // Arrange
        var request = new DeployRequest("REDIS", "test-redis", "7.4", 16379, null, null);

        // Act
        var response = service.deploy(request);

        // Assert
        await().atMost(30, SECONDS)
            .until(() -> service.getPipeline(response.id()).status() == COMPLETED);

        var pipeline = service.getPipeline(response.id());
        assertThat(pipeline.steps()).hasSize(4);
        assertThat(pipeline.steps()).allMatch(s -> s.status() == COMPLETED);
    }
}

```

Aim for at least: deploy flow, start/stop/remove lifecycle, connection string generation, and container discovery. These become your regression safety net for every phase after.

## 0.5 Set up GitHub Actions CI

yaml

```

# .github/workflows/ci.yml
name: CI

on: [push, pull_request]

jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: 'temurin'
      - name: Run backend tests
        run: cd backend && ./gradlew test --info
      - name: Upload test results
        uses: actions/upload-artifact@v4
        with:
          name: test-results
          path: backend/build/reports/tests/

  frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - name: Install and lint
        run: |
          cd frontend
          npm install
          npm run lint
          npm run build

```

## 0.6 Add CONTRIBUTING.md

Cover: local dev setup (exact commands), branch naming convention, PR expectations, how to run tests. Lack of this file is the most common reason a first-time contributor gives up before submitting a PR.

## 0.7 Add CHANGELOG.md

Start it now with v1.0.1 as the first entry. Future phases each get a version entry. This signals to users that the project is actively maintained.

---

## Phase 1 — Kafka Module (Weeks 3-5)

You already deploy Kafka containers. This phase adds actual Kafka management — topics, consumer groups, lag monitoring, and config.

### 1.1 Kafka admin client service

Spring Boot includes `spring-kafka`. Add it, then build a service that wraps `AdminClient` — the Kafka admin API.

```
java
```

```

@Service
public class KafkaAdminService {

    public List<TopicInfo> listTopics(String instanceId) {
        try (AdminClient admin = buildAdminClient(instanceId)) {
            return admin.listTopics().names().get()
                .stream()
                .map(name -> describeTopic(admin, name))
                .collect(toList());
        }
    }

    public void createTopic(String instanceId, CreateTopicRequest req) {
        try (AdminClient admin = buildAdminClient(instanceId)) {
            NewTopic topic = new NewTopic(
                req.name(),
                req.partitions(),
                (short) req.replicationFactor()
            );
            admin.createTopics(List.of(topic)).all().get();
        }
    }

    public ConsumerLagReport getConsumerLag(String instanceId, String groupId) {
        try (AdminClient admin = buildAdminClient(instanceId)) {
            // fetch offsets per partition, compare to end offsets
            // return structured lag report per topic/partition
        }
    }

    private AdminClient buildAdminClient(String instanceId) {
        KafkaInstance instance = kafkaRepository.findById(instanceId).orElseThrow();
        return AdminClient.create(Map.of(
            AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG,
            "localhost:" + instance.getBrokerPort()
        ));
    }
}

```

## 1.2 API endpoints

| Method | Endpoint                          | Description          |
|--------|-----------------------------------|----------------------|
| GET    | <code>/api/kafka/instances</code> | List Kafka instances |

| Method | Endpoint                                                      | Description                 |
|--------|---------------------------------------------------------------|-----------------------------|
| POST   | <code>/api/kafka/instances</code>                             | Deploy a new Kafka instance |
| DELETE | <code>/api/kafka/instances/{id}</code>                        | Remove instance             |
| POST   | <code>/api/kafka/instances/{id}/start</code>                  | Start                       |
| POST   | <code>/api/kafka/instances/{id}/stop</code>                   | Stop                        |
| GET    | <code>/api/kafka/instances/{id}/topics</code>                 | List topics                 |
| POST   | <code>/api/kafka/instances/{id}/topics</code>                 | Create topic                |
| DELETE | <code>/api/kafka/instances/{id}/topics/{name}</code>          | Delete topic                |
| GET    | <code>/api/kafka/instances/{id}/topics/{name}/messages</code> | Browse messages             |
| GET    | <code>/api/kafka/instances/{id}/groups</code>                 | List consumer groups        |
| GET    | <code>/api/kafka/instances/{id}/groups/{groupId}/lag</code>   | Get consumer lag            |

### 1.3 Message browser

The most-used Kafka UI feature. Use the Kafka Consumer API to read N messages from a topic and a partition/offset range:

```
java

public List<KafkaMessage> browseMessages(String instanceId, String topic,
                                          int maxMessages, long fromOffset) {
    Properties props = new Properties();
    props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, getBootstrapServers(instanceId));
    props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);
    props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class);
    props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "earliest");
    props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, false);
    props.put(ConsumerConfig.GROUP_ID_CONFIG, "port-wrangler-browser-" + UUID.randomUUID());

    try (KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props)) {
        // assign, seek, poll, return
    }
}
```

### 1.4 Consumer lag scheduler

Add a `KafkaLagScheduler` (like your existing `StatusSyncScheduler`) that polls consumer lag

every 30 seconds and stores snapshots. This powers the lag trend chart in the UI.

## 1.5 Frontend — Kafka page

Components needed:

- `KafkaInstanceCard` — status, port, version, actions
  - `TopicsTable` — sortable table with partition count, message count, size
  - `TopicCreateModal` — form: name, partitions, replication factor, retention
  - `MessageBrowser` — offset picker, message list with key/value/timestamp
  - `ConsumerGroupsPanel` — groups list with lag per topic/partition
  - `LagChart` — Recharts line chart of lag over time per group
- 

## Phase 2 — LLM Runtime Manager (Weeks 6–9)

This is the new module. By the end, Port Wrangler can deploy an Ollama instance, pull models into it, and show live metrics — all from the UI.

### 2.1 GPU detection

Add to your existing `OsDetector`:

```
java

public GpuInfo detectGpu() {
    // Check for NVIDIA
    ProcessResult nvidiaSmi = exec("nvidia-smi", "--query-gpu=name,memory.total",
                                    "--format=csv,noheader");
    if (nvidiaSmi.success()) return GpuInfo.nvidia(nvidiaSmi.output());

    // Check for AMD (Linux)
    ProcessResult rocmInfo = exec("rocminfo");
    if (rocmInfo.success()) return GpuInfo.amd();

    // Check Apple Silicon
    if (System.getProperty("os.name").contains("Mac")) {
        ProcessResult sysctl = exec("sysctl", "-n", "machdep.cpu.brand_string");
        if (sysctl.output().contains("Apple")) return GpuInfo.appleSilicon();
    }

    return GpuInfo.cpuOnly();
}
```

## 2.2 Runtime catalog

Similar to your `DatabaseCatalog`, define the supported AI runtimes:

```
java

public enum AiRuntimeType {

    OLLAMA(
        "ollama/ollama",
        11434,
        "Ollama",
        "Most popular local LLM runtime. Supports Llama, Gemma, Mistral, Qwen and 100+ models.",
        List.of("GGUF"),
        true // supports GPU
    ),
    DOCKER_MODEL_RUNNER(
        "ai/models", // OCI-based
        12434,
        "Docker Model Runner",
        "Docker's official AI model runtime. OCI-packaged models, Docker-native.",
        List.of("OCI"),
        true
    ),
    LOCAL_AI(
        "localai/localai",
        8080,
        "LocalAI",
        "OpenAI-compatible self-hosted API. Supports GGUF, GPTQ, and more.",
        List.of("GGUF", "GPTQ"),
        true
    );

    // ... fields and constructor
}
```

## 2.3 GPU-aware container configuration

When deploying an Ollama container, adjust Docker config based on detected GPU:

```
java
```

```

private HostConfig buildHostConfig(AiRuntime runtime, GpuInfo gpu) {
    HostConfig.Builder config = HostConfig.newHostConfig()
        .withPortBindings(/* ... */);

    if (runtime.isGpuEnabled() && gpu.type() != GpuType.CPU_ONLY) {
        switch (gpu.type()) {
            case NVIDIA_CUDA -> config.withDeviceRequests(List.of(
                new DeviceRequest()
                    .withCapabilities(List.of(List.of("gpu")))
                    .withCount(-1) // all GPUs
            ));
            case AMD_ROCM -> config.withDevices(List.of(
                new Device().withPathOnHost("/dev/kfd"),
                new Device().withPathOnHost("/dev/dri")
            ));
            // Apple Silicon: no special flags needed, Ollama auto-detects Metal
        }
    }
    return config.build();
}

```

## 2.4 Model pull pipeline

Model pulling is slow (gigabytes). It gets its own async pipeline, streaming progress to the frontend via SSE even before Phase 3's chat SSE:

```
java
```



```

@Service
public class ModelPullService {

    public Flux<ModelPullProgress> pullModel(String runtimeId, String modelName) {
        return Flux.create(sink -> {
            // Call Ollama's streaming pull endpoint
            // POST http://localhost:{port}/api/pull
            // Body: {"name": "llama3.2:3b", "stream": true}
            // Response is newline-delimited JSON with progress updates

            webClient.post()
                .uri("/api/pull")
                .bodyValue(Map.of("name", modelName, "stream", true))
                .retrieve()
                .bodyToFlux(String.class)
                .subscribe(
                    line -> {
                        ModelPullProgress progress = parseProgressLine(line);
                        modelPullLogRepository.save(toEntity(runtimeId, modelName, progress));
                        sink.next(progress);
                    },
                    sink::error,
                    sink::complete
                );
        });
    }
}

```

## 2.5 Model metrics polling

Poll the Ollama API for loaded model status:

```
java
```

```

@Scheduled(fixedDelay = 15000)
public void syncModelMetrics() {
    aiRuntimeRepository.findAllByStatus("RUNNING").forEach(runtime -> {
        try {
            OllamaProcessResponse ps = ollamaClient
                .get()
                .uri("http://localhost:{port}/api/ps", runtime.getHostPort())
                .retrieve()
                .bodyToMono(OllamaProcessResponse.class)
                .block(Duration.ofSeconds(5));

            ps.models().forEach(model -> {
                aiModelRepository.findByIdAndModelName(
                    runtime.getId(), model.name()
                ).ifPresent(entity -> {
                    entity.setSizeBytes(model.size());
                    entity.setIsActive(true);
                    entity.setContextWindow(model.details().contextLength());
                    aiModelRepository.save(entity);
                });
            });
        } catch (Exception e) {
            log.warn("Could not sync metrics for runtime {}", runtime.getName(), e);
        }
    });
}

```

## 2.6 New API endpoints

| Method | Endpoint                                              | Description                        |
|--------|-------------------------------------------------------|------------------------------------|
| GET    | <code>/api/ai/runtimes</code>                         | List AI runtimes                   |
| POST   | <code>/api/ai/runtimes</code>                         | Deploy a new runtime (Ollama etc.) |
| DELETE | <code>/api/ai/runtimes/{id}</code>                    | Remove runtime                     |
| POST   | <code>/api/ai/runtimes/{id}/start</code>              | Start                              |
| POST   | <code>/api/ai/runtimes/{id}/stop</code>               | Stop                               |
| GET    | <code>/api/ai/runtimes/{id}/models</code>             | List pulled models                 |
| POST   | <code>/api/ai/runtimes/{id}/models</code>             | Pull a new model (SSE stream)      |
| DELETE | <code>/api/ai/runtimes/{id}/models/{modelName}</code> | Delete model                       |

| Method | Endpoint                                   | Description                                   |
|--------|--------------------------------------------|-----------------------------------------------|
| GET    | <code>/api/ai/runtimes/{id}/metrics</code> | Live runtime metrics                          |
| GET    | <code>/api/ai/catalog</code>               | Browse available models (from Ollama library) |
| GET    | <code>/api/ai/system/gpu</code>            | GPU detection result                          |

## 2.7 Spring AI config snippet generation

Just like your connection string generator, add a config snippet generator for Spring AI integration. When a model is pulled and active, show:

```
java

public SpringAiConfig generateSpringAiConfig(AiRuntime runtime, AiModel model) {
    return switch (runtime.getRuntimeType()) {
        case OLLAMA -> new SpringAiConfig("""
            spring:
              ai:
                ollama:
                  base-url: http://localhost:%d
                  chat:
                    options:
                      model: %s
                  embedding:
                    options:
                      model: nomic-embed-text
            """).formatted(runtime.getHostPort(), model.getModelName());
        case DOCKER_MODEL_RUNNER -> new SpringAiConfig("""
            spring:
              ai:
                openai:
                  base-url: http://localhost:%d/engines
                  api-key: ignored
                  chat:
                    options:
                      model: %s
            """).formatted(runtime.getHostPort(), model.getModelName());
        // ...
    };
}
```

## 2.8 Frontend — AI Models page

Components:

- `RuntimeCard` — status, GPU badge, port, model count
  - `ModelCatalog` — searchable grid of popular models (Llama, Gemma, Mistral, Qwen, DeepSeek, Phi) with size, capability badges
  - `ModelPullDrawer` — pull progress SSE stream with animated progress bar
  - `LoadedModelsPanel` — currently loaded models with memory usage
  - `GpuInfoBanner` — detected GPU type with a "CPU only" warning if no GPU
  - `SpringAiConfigCard` — copy-ready config snippet per active model
- 

## Phase 3 — Spring AI Foundation (Weeks 10-12)

The chat panel appears. This phase builds the streaming infrastructure and the basic chat client. No tool calling or RAG yet — just a clean streaming chat UI connected to whatever model is running in your LLM runtime.

### 3.1 Spring AI auto-configuration

```
yaml
# application.yml additions
spring:
  ai:
    ollama:
      base-url: ${AI_OLLAMA_BASE_URL:http://localhost:11434}
      chat:
        options:
          model: ${AI_DEFAULT_MODEL:llama3.2:3b}
          temperature: 0.7
          num-ctx: 4096
# pgvector store config (Phase 4 prep)
vectorstore:
  pgvector:
    index-type: HNSW
    distance-type: COSINE_DISTANCE
    dimensions: 768
```

The base-url should dynamically resolve from whatever Ollama runtime is currently running in Port Wrangler itself. Add a `RuntimeAwareOllamaConfig` that reads the active runtime's port from the database on startup.

### 3.2 Chat endpoint with SSE streaming

```
java
```

```

@RestController
@RequestMapping("/api/chat")
public class ChatController {

    private final ChatClient chatClient;
    private final ChatSessionRepository sessionRepository;
    private final ChatMessageRepository messageRepository;

    @PostMapping(value = "/sessions/{sessionId}/messages",
        produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<ServerSentEvent<String>> chat(
        @PathVariable UUID sessionId,
        @RequestBody ChatRequest request) {

        // Load conversation history for context
        List<Message> history = messageRepository
            .findBySessionIdOrderByMsgOrder(sessionId)
            .stream()
            .map(this::toSpringAiMessage)
            .collect(toList());

        // Add user message to history and persist
        UserMessage userMessage = new UserMessage(request.content());
        persistMessage(sessionId, "USER", request.content());

        // Stream response
        StringBuilder fullResponse = new StringBuilder();

        return chatClient.prompt()
            .messages(history)
            .user(request.content())
            .stream()
            .content()
            .map(token -> {
                fullResponse.append(token);
                return ServerSentEvent.<String>builder()
                    .data(token)
                    .build();
            })
            .doOnComplete(() ->
                persistMessage(sessionId, "ASSISTANT", fullResponse.toString())
            );
    }

    @PostMapping("/sessions")
    public ChatSessionResponse createSession() {

```

```

        ChatSession session = new ChatSession();
        session.setTitle("New conversation");
        return toResponse(sessionRepository.save(session));
    }

    @GetMapping("/sessions")
    public List<ChatSessionResponse> listSessions() {
        return sessionRepository.findAllByOrderByUpdatedAtDesc()
            .stream().map(this::toResponse).collect(toList());
    }
}

```

### 3.3 Dynamic model switching

Users should be able to switch models mid-conversation. Expose a `ChatClientFactory` that builds a new `ChatClient` pointing to a specified model:

```

java

@Component
public class ChatClientFactory {

    private final OllamaChatModel ollamaChatModel;

    public ChatClient forModel(String modelName) {
        OllamaOptions options = OllamaOptions.builder()
            .model(modelName)
            .temperature(0.7)
            .build();

        return ChatClient.builder(ollamaChatModel)
            .defaultOptions(options)
            .defaultSystem(SYSTEM_PROMPT)
            .build();
    }
}

```

### 3.4 System prompt

The system prompt defines Port Wrangler's assistant persona. Keep it focused:

You are the Port Wrangler assistant. You help developers manage their local infrastructure – databases, Kafka, and AI models running in Docker containers.

You have access to tools that let you check status, deploy new services, read logs, and configure connections. When users ask you to take an action, use the appropriate tool rather than describing what they should do manually.

When generating configuration snippets, always use the actual ports and settings from the user's running instances, not placeholder values.

Keep responses concise. Developers prefer specific information over lengthy explanations. When something fails, show the relevant log lines.

### 3.5 Frontend — Chat panel

The chat panel sits as a sidebar or a full page, accessible from anywhere:

```
jsx
```

```
// components/chat/ChatPanel.jsx
export function ChatPanel({ sessionId }) {
  const [messages, setMessages] = useState([]);
  const [input, setInput] = useState('');
  const [isStreaming, setIsStreaming] = useState(false);

  const sendMessage = async () => {
    const userMsg = { role: 'user', content: input };
    setMessages(prev => [...prev, userMsg]);
    setInput('');
    setIsStreaming(true);

    // Add empty assistant message to stream into
    setMessages(prev => [...prev, { role: 'assistant', content: '' }]);

    const response = await fetch(`/api/chat/sessions/${sessionId}/messages`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ content: input })
    });

    const reader = response.body.getReader();
    const decoder = new TextDecoder();

    while (true) {
      const { done, value } = await reader.read();
      if (done) break;

      const chunk = decoder.decode(value);
      // Parse SSE data lines
      chunk.split('\n').forEach(line => {
        if (line.startsWith('data:')) {
          const token = line.slice(5);
          setMessages(prev => {
            const updated = [...prev];
            updated[updated.length - 1].content += token;
            return updated;
          });
        }
      });
    }
    setIsStreaming(false);
  };

  return (
    <div className="flex flex-col h-full">

```



```
<MessageList messages={messages} isStreaming={isStreaming} />
<ChatInput value={input} onChange={setInput} onSend={sendMessage}
            disabled={isStreaming} />
</div>
);
}
```

Use `react-markdown` to render assistant messages so code blocks, tables, and lists format correctly.

---

## Phase 4 — RAG Pipeline (Weeks 13-16)

Port Wrangler's data — deployment history, container logs, configurations — becomes a searchable knowledge base. The AI answers questions about your infrastructure using real data, not hallucinations.

### 4.1 Enable pgvector

Run this once against your system Postgres. Add to a Flyway migration so it's automatic on fresh installs:

```
sql
-- V4__pgvector_and_chat.sql (first lines)
CREATE EXTENSION IF NOT EXISTS vector;
```

Verify the extension is available in the Postgres 16 image you already use — it is included in the official `postgres:16` Docker image from version 16.2+.

### 4.2 Embedding model

Use `omic-embed-text` via Ollama — it's small (274MB), fast, and produces 768- dimensional embeddings. Pull it automatically when Port Wrangler's AI module starts:

```
java
```

```

@Component
public class EmbeddingModelProvisioner implements ApplicationListener<ApplicationReadyEvent> {

    @Override
    public void onApplicationEvent(ApplicationReadyEvent event) {
        // Check if an active Ollama runtime exists
        aiRuntimeRepository.findFirstByRuntimeTypeAndStatus("OLLAMA", "RUNNING")
            .ifPresent(runtime -> {
                // Pull nomic-embed-text if not already present
                if (!isModelPresent(runtime, "nomic-embed-text")) {
                    modelPullService.pullModelSync(runtime.getId(), "nomic-embed-text");
                    log.info("Pulled nomic-embed-text embedding model");
                }
            });
    }
}

```

### 4.3 Document ingestion pipeline

Define what gets indexed and when:

Automatic ingestion triggers:

| Event                     | What gets indexed                           |
|---------------------------|---------------------------------------------|
| Deployment completes      | Config (name, type, port, version), outcome |
| Container fails           | Last 100 log lines, error message           |
| Pipeline step fails       | Step type, error, container ID              |
| User adds a note (future) | Free text note                              |
| Kafka topic created       | Topic name, partition count, retention      |
| Model pulled              | Model name, runtime, size, context window   |

```

java

```

```

@Service
public class RagIngestionService {

    private final VectorStore vectorStore;
    private final EmbeddingModel embeddingModel;

    public void indexDeployment(DeploymentConfig config, PipelineStatus outcome) {
        String content = ""
            Deployment: %s (%s version %s)
            Port: %d
            Status: %s
            Deployed at: %s
            Connection string: %s
            """.formatted(
                config.getName(), config.getDbType(), config.getVersion(),
                config.getHostPort(), outcome, config.getCreatedAt(),
                connectionStringBuilder.build(config)
            );

        Document doc = new Document(content, Map.of(
            "source_type", "DEPLOYMENT",
            "instance_id", config.getId().toString(),
            "db_type", config.getDbType()
        ));

        vectorStore.add(List.of(doc));
    }

    public void indexContainerLogs(String instanceId, String instanceName,
                                    List<String> logLines) {
        // Chunk logs into 512-token windows with 50-token overlap
        List<String> chunks = chunkText(String.join("\n", logLines), 512, 50);

        List<Document> docs = IntStream.range(0, chunks.size())
            .mapToObj(i -> new Document(chunks.get(i), Map.of(
                "source_type", "CONTAINER_LOG",
                "instance_id", instanceId,
                "instance_name", instanceName,
                "chunk_index", i
            )))
            .collect(toList());

        vectorStore.add(docs);
    }
}

```

#### 4.4 RAG-augmented chat

Modify the chat endpoint to retrieve relevant context before sending to the model:

```
java
```

```

@Service
public class RagChatService {

    private final ChatClient chatClient;
    private final VectorStore vectorStore;

    public Flux<String> chat(UUID sessionId, String userMessage, List<Message> history) {

        // Retrieve relevant context
        List<Document> relevantDocs = vectorStore.similaritySearch(
            SearchRequest.builder()
                .query(userMessage)
                .topK(5)
                .similarityThreshold(0.7)
                .build()
        );

        // Build context string from retrieved docs
        String context = relevantDocs.stream()
            .map(Document::getText)
            .collect(Collectors.joining("\n\n---\n\n"));

        // Augment system prompt with context
        String augmentedSystem = """
            %s

            CURRENT INFRASTRUCTURE CONTEXT:
            The following information comes from the user's actual running instances
            and deployment history. Use it to give accurate, specific answers:

            %s
            """.formatted(BASE_SYSTEM_PROMPT, context);

        return chatClient.prompt()
            .system(augmentedSystem)
            .messages(history)
            .user(userMessage)
            .stream()
            .content();
    }
}

```

## 4.5 What RAG enables (concrete examples)

Once indexed, the assistant can answer:

- *"What port is my orders database on?"* — retrieves deployment config, answers accurately
  - *"Why did my Kafka container keep restarting last week?"* — retrieves error log chunks
  - *"What's the connection string for Redis?"* — retrieves from deployment records
  - *"Which databases have I deployed in the last month?"* — semantic search over deployment history
  - *"What models have I pulled?"* — retrieves AI model records
- 

## Phase 5 — Agentic Infrastructure Assistant (Weeks 17-20)

The AI can now take actions, not just answer questions. The user describes what they want; the assistant orchestrates the infrastructure.

### 5.1 Tool definitions

Expose your existing service layer as Spring AI tools. No new business logic — the AI goes through the same code paths as the UI:

```
java
```

@Component

```
public class InfrastructureTools {

    private final DbInstanceService dbInstanceService;
    private final KafkaAdminService kafkaAdminService;
    private final AiRuntimeService aiRuntimeService;
    private final ConnectionStringBuilder connectionStringBuilder;

    // — Database tools —————

    @Tool(description = """
        Deploy a new database instance. Supported types: POSTGRESQL, MYSQL,
        MONGODB, REDIS, MARIADB, CASSANDRA, MSSQL, CLICKHOUSE, ELASTICSEARCH,
        COUCHDB, NEO4J, DYNAMODB. Returns the instance ID and connection string.
        """)
    public DeployResult deployDatabase(
        @ToolParam(description = "Database type, e.g. POSTGRESQL") String dbType,
        @ToolParam(description = "Unique name for this instance") String name,
        @ToolParam(description = "Version tag, e.g. 16") String version,
        @ToolParam(description = "Host port to expose on localhost") int hostPort) {

        DeployRequest request = new DeployRequest(dbType, name, version,
                                                    hostPort, null, null, null);
        InstanceResponse instance = dbInstanceService.deploy(request);
        return new DeployResult(instance.id(), instance.connectionString(), instance)
    }

    @Tool(description = "List all database instances with their current status, port")
    public List<InstanceSummary> listDatabases() {
        return dbInstanceService.findAll().stream()
            .map(i -> new InstanceSummary(i.id(), i.name(), i.dbType(),
                                           i.status(), i.hostPort()))
            .collect(toList());
    }

    @Tool(description = "Get the last N lines of logs from a container. " +
        "Use this to diagnose failures or check what a service is doing")
    public String getContainerLogs(
        @ToolParam(description = "Instance ID from listDatabases") String instanceId,
        @ToolParam(description = "Number of log lines to retrieve (max 200)") int lines) {
        return dbInstanceService.getLogs(UUID.fromString(instanceId),
                                           Math.min(lines, 200));
    }

    @Tool(description = "Stop a running database instance by ID.")
    public String stopDatabase(String instanceId) {

```

```

        dbInstanceService.stop(UUID.fromString(instanceId));
        return "Stopped instance " + instanceId;
    }

    @Tool(description = "Start a stopped database instance by ID.")
    public String startDatabase(String instanceId) {
        dbInstanceService.start(UUID.fromString(instanceId));
        return "Started instance " + instanceId;
    }

    // — Kafka tools —————

    @Tool(description = "Get consumer lag for all consumer groups on a Kafka instance"
        "Returns lag per group, topic, and partition.")
    public ConsumerLagReport getKafkaConsumerLag(String kafkaInstanceId) {
        return kafkaAdminService.getAllConsumerLag(UUID.fromString(kafkaInstanceId))
    }

    @Tool(description = "Create a new Kafka topic.")
    public String createKafkaTopic(
        String kafkaInstanceId,
        String topicName,
        int partitions,
        int replicationFactor) {
        kafkaAdminService.createTopic(UUID.fromString(kafkaInstanceId),
            new CreateTopicRequest(topicName, partitions, replicationFactor));
        return "Created topic: " + topicName;
    }

    // — AI model tools —————

    @Tool(description = "List pulled AI models and whether they are currently loaded")
    public List<ModelSummary> listAiModels() {
        return aiRuntimeService.getAllModels().stream()
            .map(m -> new ModelSummary(m.getModelName(), m.getIsActive(),
                m.getSizeBytes(), m.getRuntimeName()))
            .collect(toList());
    }

    @Tool(description = "Generate a ready-to-use Spring AI configuration for a model"
        "Returns YAML to paste into application.yml.")
    public String getSpringAiConfig(String modelName) {
        return aiRuntimeService.generateSpringAiConfig(modelName);
    }

    // — System tools —————

```



```

@Tool(description = "Get an overall health summary of all running infrastructure
                    "database instances, Kafka instances, and AI models.")
public InfrastructureSummary getInfrastructureSummary() {
    return InfrastructureSummary.of(
        dbInstanceService.getStats(),
        kafkaAdminService.getStats(),
        aiRuntimeService.getStats()
    );
}
}

```

## 5.2 Wire tools into the chat client

```

java

@Configuration
public class ChatClientConfig {

    @Bean
    public ChatClient chatClient(OllamaChatModel model,
                                InfrastructureTools tools) {
        return ChatClient.builder(model)
            .defaultSystem(SYSTEM_PROMPT)
            .defaultTools(tools)    // Spring AI scans @Tool methods automatically
            .build();
    }
}

```

That's it. Spring AI handles the tool calling loop — it sends tool definitions to the model, receives tool call requests, executes the Java methods, and sends results back for the model to generate the final response.

## 5.3 Tool call visibility in the UI

Show users what tools the AI called. This is important for trust and debuggability.

When the backend streams a response, emit tool call events as special SSE types:

```

java

// SSE event types
// data: {token}                <- regular token
// event: tool_call\ndata: {json} <- tool being called
// event: tool_result\ndata: {json} <- tool result received
// event: done\ndata: [DONE]      <- stream complete

```

In the frontend, render tool calls as expandable cards inside the message:



```
Called: listDatabases()
Result: [PostgreSQL "orders-db" RUNNING :5432]
```

## 5.4 Agent safety guardrails

Tool calling that takes real actions needs safeguards:

**Confirmation for destructive actions.** Before executing `stopDatabase` or `deleteKafkaTopic`, emit a special SSE event type `confirmation_required`. The frontend renders a confirm/cancel prompt. The backend holds the Flux open waiting for the user's response via a follow-up message.

```
java
// In InfrastructureTools
@Tool(description = "Remove a database instance permanently.")
public String removeDatabase(String instanceId) {
    // This method is only called after user confirmation
    // The confirmation flow is handled in the chat controller
    dbInstanceService.remove(UUID.fromString(instanceId));
    return "Removed instance " + instanceId;
}
```

**Read-only mode.** Add a global setting in UI (toggle in header) that disables all state-changing tools. In read-only mode the AI can answer questions but cannot deploy, stop, or modify anything. Implement by checking a flag in the tool methods.

---

## Phase 6 — Polish & Community Launch (Weeks 21-24)

Features are complete. This phase is about making the project discoverable, trustworthy, and contributor-friendly.

### 6.1 Documentation site

Use **Docusaurus** (or a simple Vite + MDX setup if you prefer to stay in the existing stack). Host it on GitHub Pages for free.

Structure:

```
docs/
├── getting-started/
│   ├── installation.md
│   ├── first-database.md
│   ├── first-kafka.md
│   └── first-ai-model.md
├── features/
│   ├── database-management.md
│   ├── kafka-management.md
│   ├── ai-runtime-manager.md
│   ├── ai-assistant.md
│   └── rag-pipeline.md
├── configuration/
│   ├── environment-variables.md
│   ├── gpu-setup.md
│   └── spring-ai-integration.md
└── contributing/
    ├── development-setup.md
    ├── architecture.md
    └── adding-a-database.md
```

## 6.2 Demo video

A 3-5 minute screen recording covering:

1. One-click deploy of Postgres + Redis + Kafka (30 sec)
2. Pull a Llama model, watch progress bar, see GPU stats (45 sec)
3. Chat with the assistant: *"Set up my orders service stack"* — watch it deploy (60 sec)
4. Ask *"Why is my Kafka consumer lagging?"* — RAG returns real log data (45 sec)
5. Copy the generated Spring AI config into an IDE (30 sec)

Upload to YouTube, embed in README, link from docs site. This is the single most important thing for GitHub stars.

## 6.3 GitHub community files

```
.github/
├── ISSUE_TEMPLATE/
│   ├── bug_report.md
│   ├── feature_request.md
│   └── add_database.md          # template for requesting new DB support
├── PULL_REQUEST_TEMPLATE.md
└── CONTRIBUTING.md
```

## 6.4 Labels and milestones

Set up GitHub labels before the first external contributors arrive:

- `good first issue` — small, well-scoped, documented
- `help wanted` — medium complexity, needs contributor
- `phase-X` — tracks which roadmap phase an issue belongs to
- `database-request` — requests for new database support
- `ai` — Spring AI / RAG related
- `bug`, `enhancement`, `documentation`, `question`

Tag 5-10 existing issues or create new ones as `good first issue`. These are the entry point for contributors. Without them, interested developers have no way in.

## 6.5 Release cadence

- **Major releases** — one per phase completion (v2.0 = Kafka, v3.0 = LLM manager, etc.)
- **Minor releases** — bug fixes and small additions within a phase
- **Changelog** — every release has a section in CHANGELOG.md

Use GitHub Releases with release notes generated from the changelog section.

The native installer builds (macOS .dmg, Windows .exe) attach to each major release.

## 6.6 ProductHunt and community posting

When Phase 5 is complete, post to:

- **ProductHunt** — requires a proper tagline, description, GIF, and 5-10 upvoters on launch day. Line these up in advance.
  - **r/programming, r/java, r/selfhosted** — community posts, not promotional. Lead with the technical architecture, not the product pitch.
  - **Hacker News Show HN** — highest-value for developer tools. Time it for a weekday morning US time.
  - **Dev.to article** — write a technical deep-dive on the Spring AI + tool calling implementation. Link to the repo at the end. Technical articles drive sustained traffic.
-

## 12. API Design Reference

### Versioning

All new endpoints go under `/api/v2/` to keep backward compatibility. Existing `/api/` endpoints stay unchanged. Document both versions.

### Consistent response envelope (new endpoints)

```
json

{
  "data": { ... },
  "meta": {
    "timestamp": "2026-06-08T10:30:00Z",
    "requestId": "uuid"
  },
  "errors": []
}
```

### Pagination for list endpoints

All list endpoints that can grow large use cursor-based pagination:

```
GET /api/v2/kafka/instances/{id}/topics?limit=20&cursor=eyJpZCI6MTAwfQ
```

Response includes `nextCursor` when more results exist.

### SSE event format (streaming endpoints)

```
Content-Type: text/event-stream

event: token
data: Hello

event: token
data: , world

event: tool_call
data: {"tool":"listDatabases","args":{}}

event: tool_result
data: {"tool":"listDatabases","result":[...]}

event: done
data: [DONE]
```

## 13. Frontend Architecture

### State management split

| Zustand (client UI state)       | React Query (server state) |
|---------------------------------|----------------------------|
| └─ chatPanelOpen: boolean       | └─ useInstances()          |
| └─ activeChatSession: UUID      | └─ useKafkaInstances()     |
| └─ selectedModelForChat: string | └─ useAiRuntimes()         |
| └─ confirmationPending: {...}   | └─ useModels(runtimeId)    |
| └─ readOnlyMode: boolean        | └─ useChatSessions()       |
|                                 | └─ useContainerMetrics(id) |

### Polling intervals by data type

| Data               | Interval  | Reason                              |
|--------------------|-----------|-------------------------------------|
| Container status   | 10s       | Already your existing interval      |
| Kafka consumer lag | 30s       | Lag changes slowly                  |
| AI model metrics   | 15s       | Inference state changes mid-request |
| Chat sessions list | on-demand | User-triggered                      |

### Page routing

|                |                                                    |
|----------------|----------------------------------------------------|
| /              | → Dashboard (stats overview)                       |
| /deploy        | → Deploy workspace (existing)                      |
| /instances/:id | → Instance detail                                  |
| /kafka         | → Kafka instances list                             |
| /kafka/:id     | → Kafka instance detail (topics, groups, messages) |
| /ai            | → AI runtimes and models                           |
| /ai/:runtimeId | → Runtime detail (models, metrics)                 |
| /chat          | → Full-page chat                                   |
| /images        | → Image management (existing)                      |
| /settings      | → Configuration                                    |

## 14. Testing Strategy

### Unit tests (fast, no Docker)

- `ConnectionStringBuilderTest` — verify connection strings for all 14 DB types

- `DatabaseCatalogTest` — catalog completeness and defaults
- `GpuDetectorTest` — mock process execution, verify GPU type parsing
- `RagIngestionServiceTest` — chunking logic, metadata tagging
- `InfrastructureToolsTest` — mock service layer, verify tool method contracts

## Integration tests (Testcontainers)

- `DeploymentPipelineIntegrationTest` — full 4-step pipeline against real Docker
- `KafkaAdminIntegrationTest` — topic CRUD, consumer group queries
- `ModelPullIntegrationTest` — mock Ollama HTTP server (WireMock), verify pull flow
- `RagSearchIntegrationTest` — index documents, query, verify semantic retrieval
- `ChatStreamingIntegrationTest` — mock ChatModel, verify SSE stream format

## End-to-end tests (Playwright)

Add a Playwright test suite covering the critical happy paths:

javascript

```
// e2e/deploy-database.spec.js
test('deploys postgres and shows connection string', async ({ page }) => {
  await page.goto('http://localhost:5173');
  await page.click('[data-testid="deploy-nav"]');
  await page.click('[data-testid="db-POSTGRESQL"]');
  await page.fill('[data-testid="instance-name"]', 'test-postgres');
  await page.fill('[data-testid="host-port"]', '5499');
  await page.click('[data-testid="deploy-button"]');

  // Pipeline should complete
  await expect(page.locator('[data-testid="pipeline-status"]'))
    .toHaveText('COMPLETED', { timeout: 30000 });

  // Connection string should be visible
  await expect(page.locator('[data-testid="connection-string"]'))
    .toContainText('postgresql://');
});
```

## Test data builders

Use the builder pattern for test data — avoids brittle test setup:

java

```
public class DeployRequestBuilder {  
    public static DeployRequest postgres() {  
        return new DeployRequest("POSTGRESQL", "test-" + randomAlpha(6),  
                                   "16", randomPort(), "admin", "secret", "testdb");  
    }  
    public static DeployRequest redis() {  
        return new DeployRequest("REDIS", "test-" + randomAlpha(6),  
                                   "7.4", randomPort(), null, null, null);  
    }  
}
```

---

## 15. CI/CD Pipeline

### Full pipeline (all phases)

yaml



```
# .github/workflows/ci.yml
name: CI/CD

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
  release:
    types: [created]

jobs:
  test-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '21', distribution: 'temurin' }
      - name: Unit tests
        run: cd backend && ./gradlew test
      - name: Integration tests
        run: cd backend && ./gradlew integrationTest
      - name: Test coverage report
        run: cd backend && ./gradlew jacocoTestReport
      - name: Upload coverage
        uses: codecov/codecov-action@v4

  test-frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: cd frontend && npm ci
      - run: cd frontend && npm run lint
      - run: cd frontend && npm run test
      - run: cd frontend && npm run build

  e2e:
    runs-on: ubuntu-latest
    needs: [test-backend, test-frontend]
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4
      - name: Start full stack
        run: docker compose up -d --build
```

```
- name: Wait for health
  run: |
    timeout 120 bash -c 'until curl -f http://localhost:8080/actuator/health;
- name: Run Playwright tests
  run: cd e2e && npm ci && npx playwright test
```

#### docker-publish:

```
runs-on: ubuntu-latest
needs: [e2e]
if: github.event_name == 'release'
steps:
- uses: actions/checkout@v4
- name: Build and push Docker image
  run: |
    docker build -t ghcr.io/${{ github.repository }}:${{ github.ref_name }} .
    docker push ghcr.io/${{ github.repository }}:${{ github.ref_name }}
    docker tag ... :latest
    docker push ... :latest
```

#### native-installers:

```
runs-on: ${{ matrix.os }}
needs: [e2e]
if: github.event_name == 'release'
strategy:
  matrix:
    os: [macos-latest, windows-latest]
steps:
- uses: actions/checkout@v4
- uses: actions/setup-java@v4
  with: { java-version: '21', distribution: 'temurin' }
- name: Build installer
  run: cd backend && ./gradlew jpackageInstaller
- name: Upload release asset
  uses: softprops/action-gh-release@v2
  with:
    files: backend/build/dist/*
```

---

## 16. Resume Showcase Guide

### How to present each phase in interviews

#### Phase 0 (Flyway + Tests)

"Before extending the project I replaced Hibernate's ddl-auto with Flyway migrations and wrote Testcontainers-based integration tests for the core

deployment pipeline. This gave me a reliable regression safety net and demonstrated database migration discipline."

### Phase 1 (Kafka)

"I built a Kafka management module using the AdminClient API — topic CRUD, consumer group monitoring, real-time lag tracking with a time-series chart, and a message browser with offset navigation."

### Phase 2 (LLM Runtime)

"I implemented GPU detection across NVIDIA CUDA, AMD ROCm, and Apple Silicon and used it to dynamically configure Docker container flags for optimal inference performance. I also built a streaming model pull pipeline using Project Reactor's Flux and SSE."

### Phase 3 (Spring AI + Streaming)

"I integrated Spring AI 1.0 to build a streaming chat client backed by a locally-deployed Ollama instance, using Server-Sent Events and Spring WebFlux. Chat history is persisted and re-sent as context on each turn."

### Phase 4 (RAG)

"I implemented a RAG pipeline using pgvector (same Postgres instance, no extra service) and Ollama's nomic-embed-text embedding model. Deployment configs, container logs, and Kafka topic metadata are chunked, embedded, and indexed automatically. Retrieval uses cosine similarity with a configurable threshold."

### Phase 5 (Tool Calling / Agents)

"I exposed the existing service layer as Spring AI @Tool methods so the AI assistant can deploy containers, manage Kafka topics, and pull models — the same code paths as the UI, with no new business logic. I added a confirmation flow for destructive actions and a read-only mode for safety."

### Skills demonstrated by the project (for your CV)

Spring Boot 3 · Spring AI 1.0 · Tool Calling · RAG Pipelines  
pgvector · Vector Embeddings · Semantic Search  
Server-Sent Events · Project Reactor / WebFlux  
Docker Java SDK · Container Lifecycle Management  
Apache Kafka AdminClient · Consumer Lag Monitoring  
Flyway Database Migrations · Testcontainers  
React 19 · React Query · Zustand · Recharts  
GitHub Actions CI/CD · Native Installer Distribution

## What makes this stand out versus other portfolio projects

Most portfolio projects are CRUD apps or tutorial follow-alongs. This project demonstrates:

- 1. **Production engineering decisions** — Flyway over ddl-auto, Testcontainers over mocks, SSE over polling, pgvector over a separate vector DB service
- 2. **Integration depth** — Docker daemon, Kafka AdminClient, Ollama API, Spring AI, and pgvector all in one coherent system
- 3. **Early technology adoption** — Spring AI 1.0 is very new. Tool calling and agentic patterns are the current frontier of applied AI engineering
- 4. **Open source discipline** — CONTRIBUTING.md, issue templates, changelog, CI/CD pipeline, release automation — these signal engineering maturity
- 5. **A real problem solved** — every Java developer who runs services locally is the target user. Interviewers understand the problem immediately

## Appendix: Recommended Model Catalog

When users browse models to pull in the UI, surface these as pre-configured options:

| Model            | Tag              | Size   | Best for                   | RAM needed |
|------------------|------------------|--------|----------------------------|------------|
| Llama 3.2        | llama3.2:3b      | 2.0 GB | General chat, coding       | 4 GB       |
| Llama 3.2        | llama3.2:1b      | 1.3 GB | Fast responses, low RAM    | 2 GB       |
| Gemma 3          | gemma3:4b        | 3.3 GB | Instruction following      | 6 GB       |
| Qwen 3           | qwen3:8b         | 5.2 GB | Code, reasoning            | 8 GB       |
| DeepSeek-R1      | deepseek-r1:8b   | 4.9 GB | Step-by-step reasoning     | 8 GB       |
| Mistral          | mistral:7b       | 4.1 GB | Balanced general purpose   | 8 GB       |
| Phi-4            | phi4:14b         | 9.1 GB | Strong reasoning, mid-size | 16 GB      |
| nomic-embed-text | nomic-embed-text | 274 MB | RAG embeddings (required)  | 1 GB       |

Good luck. The project is genuinely good — this roadmap takes it from a useful developer tool to a showcase of modern Java and AI engineering.